

Codex + Obsidian 模块化知识系统

模块开发规范

文档版本：0.1.0

状态：Draft

适用对象：模块开发者、Prompt 设计者、业务 workflow 开发者

依赖文档：《核心平台接口规范》

1. 文档目的

本文档规定业务模块的目录结构、Manifest、Schema、Prompt、工作流、审核策略、事件接口和测试方法。

一个合格模块必须做到：

- 不修改核心平台代码即可安装；
 - 不依赖其他模块的内部文件结构；
 - 只处理自己拥有的文件；
 - 通过标准接口生成操作计划；
 - 将不确定事项提交到公共 Review Queue；
 - 向 Today Dashboard 提供标准项目；
 - 支持暂停、完成和归档；
 - 可以通过实例复用；
 - Prompt、Schema 和确定性规则相互分离。
-

2. 模块不是单个 Prompt

一个完整模块通常包括：

```
业务 Schema
+ Prompt
+ 文件目录规则
+ 文件命名规则
+ 工作流
+ 审核策略
+ Dashboard Provider
+ 事件订阅与发布
+ 模板
+ 测试
```

Prompt只负责需要语义理解的部分。

以下事项不应依赖 Prompt:

- 日期计算;
- 文件 Hash;
- 作业编号标准化;
- 周报到期判断;
- 文件移动;
- YAML格式校验;
- 状态机合法性;
- 重复文件检测。

3. 模块分类

3.1 配置型模块

主要包含:

- Schema;
- Prompt;
- 模板;
- 目录规则;
- 审核策略。

适用于:

- 日记;
- 阅读记录;
- 会议记录;
- 灵感记录。

3.2 工作流型模块

在配置型基础上增加:

- 状态机;
- 周期任务;
- 多步骤处理流程;
- 长期档案更新。

适用于:

- 课程;
 - 实习;
 - 申请;
 - 求职;
 - 项目管理。
-

3.3 集成型模块

增加外部系统适配器，例如：

- Gmail;
- Google Calendar;
- GitHub;
- 学校学习平台;
- 财务系统。

集成型模块不得绕过核心权限和日志。

4. 标准模块目录

```
90-System/Modules/{module_id}/
├── module.yaml
├── README.md
├──
├── schemas/
├── prompts/
├── workflows/
├── rules/
├── templates/
├── dashboard/
├── scripts/
├── migrations/
├── tests/
│   ├── fixtures/
│   ├── expected/
│   └── cases.yaml
```

只有 `module.yaml` 是强制文件。

其他目录按能力选择，但模块声明的文件必须真实存在。

5. module_id 命名规则

`module_id` MUST:

- 使用小写;
- 使用英文;
- 使用短横线;
- 保持稳定;
- 不包含具体年份或实例名称。

正确：

```
course
experience-log
application-tracker
job-search
project-management
```

错误:

```
澳洲申请
monash-2027
my-internship
CourseModule
```

6. module.yaml 规范

完整示例:

```
id: experience-log
name: 经历记录
version: 1.0.0
status: enabled

description: >
  管理实习、工作、科研助理和其他长期经历记录，
  生成周期总结、技能证据、STAR素材和简历素材。

engine:
  min_version: 0.1.0
  max_version: 0.x

module_type: workflow

capabilities:
  - capture-processing
  - metadata-enrichment
  - file-organization
  - link-suggestion
  - periodic-summary
  - dashboard-items
  - review-items
  - event-publishing

accepted_inputs:
  - markdown
  - text
  - pdf
```

- image

inbox:

module_level:

enabled: true

path: 20-Workspace/Experiences/Inbox

instance_level:

enabled: true

path_pattern: "{instance_root}/Inbox"

allow_global_routing: true

routing:

matcher: workflows/match.yaml

auto_route_threshold: 0.85

review_threshold: 0.60

entry_workflows:

capture: workflows/process-capture.yaml

weekly: workflows/weekly-rollup.yaml

monthly: workflows/monthly-rollup.yaml

finalize: workflows/finalize-instance.yaml

archive: workflows/archive-instance.yaml

review_resolve: workflows/review-resolve.yaml

schemas:

instance: schemas/instance.schema.yaml

daily_log: schemas/daily-log.schema.yaml

weekly_review: schemas/weekly-review.schema.yaml

evidence: schemas/evidence.schema.yaml

rules:

paths: rules/paths.yaml

naming: rules/naming.yaml

linking: rules/linking.yaml

review: rules/review-policy.yaml

reading: rules/reading-policy.yaml

dashboard:

provider: dashboard/today-provider.yaml

events:

publishes:

- evidence.created

- summary.generated

- instance.archived

subscribes:

- file.added

```
dependencies:
  components:
    - periodic-rollup
    - evidence-extractor
    - attachment-sidecar

scheduled_jobs:
  - id: weekly-review
    scope: instance
    trigger:
      type: weekly
      day: sunday
    workflow: workflows/weekly-rollup.yaml

permissions:
  max_default_read_level: 0
  allow_external_network: false
  allow_delete: false
  allow_bulk_move: false
```

7. README 要求

每个模块 SHOULD 包含 README，至少说明：

1. 模块解决什么问题；
2. 支持哪些实例；
3. 输入放在哪里；
4. 自动完成哪些操作；
5. 哪些操作需要审核；
6. 会生成哪些文件；
7. 如何暂停和归档；
8. 需要哪些共享组件；
9. 有哪些已知限制；
10. 最小使用示例。

8. 实例 Schema

每个模块必须定义实例最小结构。

例如 `experience-log`：

```
type: object
required:
  - instance_id
  - module
```

```
- status
- content_root
- experience_type

properties:
  instance_id:
    type: string

  module:
    const: experience-log

  status:
    enum:
      - planned
      - active
      - paused
      - completed
      - archived

  experience_type:
    enum:
      - internship
      - employment
      - research
      - volunteering
      - other

  organization:
    type: string

  role:
    type: string

  start_date:
    type:
      - string
      - "null"

  end_date:
    type:
      - string
      - "null"
```

业务模块不能要求用户频繁手动维护实例 YAML。模块应通过工作流自动补充可推导字段。

9. 业务 Schema 设计原则

每种长期文档类型 SHOULD 有独立 Schema，例如：

```
daily-log
weekly-review
application-program
lecture-note
assignment
journal-entry
```

Schema 字段分为：

9.1 必填字段

系统无法正常处理时必须存在。

9.2 自动字段

由模块或核心平台生成。

9.3 可选字段

缺失不影响主要功能。

9.4 待确认字段

可以为空，并通过 Review Queue 补充。

示例：

```
due_date:
  type:
    - string
    - "null"
  x-review-if-missing: false

related_assignment:
  type:
    - string
    - "null"
  x-review-if-confidence-below: 0.75
```

10. Prompt 分层规范

模块 Prompt 应拆分为小任务，不应使用一个巨大 Prompt 完成全部工作。

建议：


```
prompts/
├── classify.md
├── normalize.md
├── extract-metadata.md
├── suggest-links.md
├── compare-update.md
├── weekly-review.md
└── final-summary.md
```

10.1 Prompt 必须包含的内容

每个 Prompt 至少说明：

1. 任务目标；
2. 输入文件类型；
3. 可使用的上下文；
4. 禁止行为；
5. 输出 Schema；
6. 置信度要求；
7. 何时创建审核项；
8. 哪些内容不得编造。

10.2 Prompt 不得包含

- 另一个模块的内部目录；
- 全局固定用户信息；
- Git命令；
- 未授权文件删除；
- 对整个 Vault 的无限搜索；
- 默认联网搜索；
- 直接输出无法验证的事实。

10.3 Prompt 示例

任务

将用户的零散经历记录整理为结构化日报。

允许使用

- 当前 Capture；
- 当前经历实例配置；
- 最近 7 天该实例的日报标题；
- 当前项目和技能索引。

不允许

- 编造未出现的工作成果；
- 编造数字；
- 将计划写成已完成事项；
- 修改用户表达的事实；
- 读取其他实例。

输出

返回符合 process-result.schema.json 的 JSON：

- suggested_title
- frontmatter_patch
- normalized_sections
- link_suggestions
- review_items
- evidence_candidates

审核规则

以下情况必须生成审核项：

- 无法判断记录日期；
- 无法判断所属项目；
- 推断出量化成果；
- 建议关联其他模块。

11. Prompt 输出必须结构化

模块不能依赖从自由文本中猜测操作。

Codex输出示例：

```
{
  "suggested_title": "2027-03-08 实习记录",
  "frontmatter_patch": {
    "type": "daily-log",
    "date": "2027-03-08"
  },
  "sections": {
    "work": [
      "排查登录接口异常"
    ],
    "problems": [
      "Refresh Token 更新逻辑异常"
    ]
  }
}
```

```

],
"learning": [
  "理解 Token 刷新流程"
],
},
"link_suggestions": [
  {
    "target": "[Authentication Module]",
    "confidence": 0.91
  }
],
"review_items": []
}

```

workflows will convert this result into an Operation Plan.

12. workflow规范

workflow文件描述:

- 输入;
- 执行步骤;
- 使用哪个 Prompt;
- 使用哪些确定性组件;
- 输出;
- 错误策略;
- 审核策略。

示例:

```

id: process-capture
version: 1

inputs:
- capture
- instance

steps:
- id: validate-input
  uses: core.validate-capture

- id: classify
  uses: codex.prompt
  with:
    prompt: prompts/classify.md

- id: normalize
  uses: codex.prompt

```

```
with:
  prompt: prompts/normalize.md

- id: validate-output
  uses: core.validate-schema
  with:
    schema: schemas/daily-log.schema.yaml

- id: build-plan
  uses: core.build-operation-plan

- id: submit-review
  uses: core.review-filter

outputs:
  - operation_plan
  - events
```

13. workflow 设计原则

workflow SHOULD:

- 每一步职责单一;
- 支持失败重试;
- 支持幂等;
- 不隐藏文件修改;
- 在关键步骤验证 Schema;
- 不直接调用另一个模块的 workflow;
- 通过事件与其他模块协作;
- 在调用 Codex 前尽量由规则缩小上下文。

14. Match 接口开发规范

模块必须实现 `match`。

匹配依据优先级:

```
模块或实例路径
> Frontmatter
> 文件名
> 活跃实例
> 文本预览
> 完整正文
```

模块不应为了分类直接读取整份附件。

输出：

```
{
  "matched": true,
  "confidence": 0.88,
  "suggested_instance": "company-internship-2027",
  "suggested_type": "daily-log",
  "reasons": [
    "位于 Experiences 模块 Inbox",
    "正文出现工作记录"
  ],
  "required_read_level": 0
}
```

15. 目录规则规范

`rules/paths.yaml` 定义模块数据布局。

```
instance_root: "20-Workspace/Experiences/{instance_id}"

folders:
  inbox: Inbox
  daily: Daily
  weekly: Weekly
  monthly: Monthly
  projects: Projects
  evidence: Evidence
  summaries: Summaries

type_routes:
  daily-log: Daily
  weekly-review: Weekly
  monthly-review: Monthly
  project-note: Projects
```

路径规则中不得硬编码单个实例名称。

16. 命名规则规范

```
patterns:
  daily-log: "{date}"
  weekly-review: "{year}-W{week:02}-Review"
  monthly-review: "{year}-{month:02}-Review"
```

```
project-note: "{project_slug}"
```

```
numbering:
```

```
width: 2
```

命名规则 SHOULD:

- 稳定;
- 可预测;
- 对 Obsidian 友好;
- 避免依赖 AI 自由发挥;
- 在重命名时更新链接。

17. 附件读取策略

模块必须声明默认读取政策。

```
default_level: 0
```

```
rules:
```

```
- type: assignment-specification
```

```
allow:
```

```
- 0
```

```
- 1
```

```
- 2
```

```
require_user_for:
```

```
- 3
```

```
- type: lecture-slides
```

```
allow:
```

```
- 0
```

```
- 1
```

```
require_user_for:
```

```
- 2
```

```
- 3
```

模块请求 Level 2 或 Level 3 时必须说明原因。

18. 双向链接规则

`rules/linking.yaml` 示例:

```
limits:
```

```
max_auto_links_per_note: 6
```

```
max_review_links_per_note: 5
```

```
auto:
```

```
- from: daily-log  
  to: instance-dashboard  
  confidence_min: 1.0
```

```
- from: weekly-review  
  to: daily-log  
  confidence_min: 0.95
```

```
review:
```

```
- from: lecture-note  
  to: assignment  
  confidence_below: 0.85
```

```
forbidden:
```

```
- from: journal-entry  
  to: person-profile  
  unless_user_confirmed: true
```

模块生成链接建议，核心平台负责执行。

19. Review Policy 规范

```
auto_execute:
```

```
- add-instance-parent-link  
- add-created-date  
- create-sidecar  
- normalize-new-file-name
```

```
review_after_execute:
```

```
- enrich-long-term-summary  
- generate-resume-material  
- infer-secondary-topic
```

```
review_before_execute:
```

```
- delete-file  
- merge-files  
- bulk-move  
- overwrite-user-section  
- update-critical-status
```

```
confidence:
```

```
  auto_min: 0.90  
  review_min: 0.60
```

模块必须明确哪些字段属于关键业务字段。

例如申请模块中的：

- `application_status` ;
- `deadline` ;
- `tuition` ;
- `english_requirement` ;

不应按照普通字段处理。

20. 用户原文保护规则

模块必须定义哪些区域属于：

```
user-owned
ai-managed
system-managed
mixed
```

例如日记模块：

```
sections:
  原始日记:
    owner: user
    ai_edit: forbidden

  系统整理:
    owner: ai
    ai_edit: allowed

  待确认:
    owner: mixed
    ai_edit: allowed
```

实习模块可以允许对原文进行轻微格式整理，但不得改变事实。

21. Dashboard Provider 规范

模块返回标准 Dashboard Item，不直接修改 `Today.md` 。

```
provider_id: experience-log-today

queries:
- id: pending-weekly-review
  condition:
```



```
instance_status: active
weekly_review_missing: true
```

```
output:
  category: summary
  priority: medium
  title: "本周经历总结尚未生成"
  actions:
    - generate
    - defer
```

模块不得将所有信息都放进 Today。

只展示：

- 到期；
- 异常；
- 待用户行动；
- 有实际价值的进度。

22. 周期任务规范

模块可以声明：

```
scheduled_jobs:
  - id: weekly-rollup
    scope: instance
    trigger:
      type: weekly
      day: sunday
      time: "18:00"
    workflow: workflows/weekly-rollup.yaml
    catch_up: true
```

`catch_up: true` 表示电脑离线时，下次启动补做。

周期任务不应直接通知用户“无变化”的内容，除非模块明确需要。

23. 总结层级规范

模块 SHOULD 使用分层总结：

```
原始记录
→ 周总结
```

- 月总结
- 阶段总结
- 最终总结

上层总结 SHOULD 优先读取下一级总结。

只有需要核对细节时才回读原始记录。

每份总结应保存：

- 时间范围；
- 输入文件；
- 生成时间；
- 使用 Prompt 版本；
- 用户是否审核；
- 后续总结引用。

24. 事件发布规范

模块发布事件时必须：

- 使用标准事件名称；
- 指明来源模块和实例；
- 提供实体 ID；
- 避免携带整篇文档；
- 只传递必要摘要和引用。

示例：

```
{
  "event_type": "summary.generated",
  "source_module": "experience-log",
  "source_instance": "company-internship-2027",
  "entity": {
    "type": "weekly-review",
    "id": "EXP-WEEK-2027-10"
  },
  "payload": {
    "file": "[[2027-W10 实习周报]]",
    "period_start": "2027-03-01",
    "period_end": "2027-03-07"
  }
}
```

25. 事件订阅规范

模块只能通过事件订阅另一个模块的输出。

禁止：

- 直接扫描另一个模块全部目录；
- 直接修改另一个模块的 YAML；
- 依赖另一个模块的私有文件命名；
- 导入另一个模块内部脚本。

订阅声明：

```
events:
  subscribes:
    - evidence.created
```

事件处理器：

```
workflows/on-evidence-created.yaml
```

26. 模块实例生命周期

模块必须实现或使用公共组件处理：

```
planned
→ active
→ paused
→ completed
→ archived
```

Active

- 接收 Inbox；
- 运行周期任务；
- 显示 Dashboard Item。

Paused

- 默认不自动处理；
- 不运行周期任务；
- 可以手动调用。

Completed

- 停止新增业务；
- 生成最终总结；
- 等待用户确认归档。

Archived

- 不生成提醒；
- 不主动处理；
- 保留搜索和链接；
- 保留数据和日志。

27. 归档 workflow

模块归档实例时 SHOULD：

1. 检查未处理 Inbox；
2. 检查 Pending Review；
3. 检查未完成周期总结；
4. 生成最终总结；
5. 更新实例 Dashboard；
6. 停止调度任务；
7. 将实例状态设为 archived；
8. 发布 `instance.archived`；
9. 不删除历史数据。

28. 模块依赖规范

模块可以依赖共享组件：

```
dependencies:  
components:  
  - periodic-rollup  
  - attachment-sidecar
```

模块不应直接依赖业务模块。

确有依赖时，必须：

- 明确版本范围；
- 允许依赖模块不存在时降级；
- 不导致核心平台无法启动。

29. 可选脚本规范

模块脚本位于：

```
scripts/
```

脚本适用于：

- 特殊编号转换；
- 特定数据格式解析；
- 模块专属确定性计算。

脚本必须：

- 接收结构化输入；
- 输出结构化结果；
- 不直接任意修改 Vault；
- 不访问未授权目录；
- 不默认联网；
- 可被单元测试。

30. 模块测试结构

```
tests/
├── fixtures/
│   ├── simple-capture/
│   ├── ambiguous-capture/
│   └── attachment/
├── expected/
│   ├── simple-plan.json
│   ├── ambiguous-review.json
│   └── attachment-sidecar.md
└── cases.yaml
```

31. 必须测试的场景

匹配

- 明确实例 Inbox；
- 模块 Inbox；
- 全局 Inbox；

- 不属于本模块；
- 多模块竞争。

处理

- 正常输入；
- 字段缺失；
- 用户写得混乱；
- 文件名错误；
- 重复处理。

权限

- 自动操作；
- 黄色审核；
- 红色阻止。

Prompt

- 不编造事实；
- 不修改用户原意；
- 输出合法 JSON；
- 低置信度生成 Review Item。

生命周期

- Active；
- Paused；
- Completed；
- Archived。

32. Prompt 回归测试

Prompt修改后必须使用固定 Fixture 测试。

例如输入：

今天继续搞登录，refresh token 有问题，
后面和同事一起改好了。

期望必须满足：

- 提取登录和 Refresh Token；
- 不编造性能提升；
- 不编造使用的技术栈；
- 不把“改好了”扩展成具体未出现的代码方案；
- 生成合理的技能候选；

- 不创建无链接。

33. 模块验收标准

一个模块只有满足以下条件才可标记为 `enabled`：

1. `module.yaml` 合法；
2. 与核心版本兼容；
3. 所有声明文件存在；
4. Schema验证通过；
5. 至少有一个正常 Fixture；
6. 至少有一个模糊输入 Fixture；
7. 操作计划中不存在未授权操作；
8. Prompt输出可结构化解析；
9. 支持实例暂停和归档；
10. 不包含其他实例的硬编码路径；
11. 不要求用户频繁手写 YAML；
12. 不默认完整读取附件。

34. 新模块开发流程

第一步：定义场景边界

明确：

- 模块管理什么；
- 不管理什么；
- 核心实体有哪些；
- 哪些内容属于其他模块。

第二步：定义实例

确认该模块是否需要多个实例。

例如：

```
course: 需要
journal: 通常不需要
job-search: 可以按求职周期创建
```

第三步：设计 Schema

先定义数据，再写 Prompt。

第四步：设计目录和 Inbox

确定：

- 模块 Inbox；
- 实例 Inbox；
- 明确投放目录；
- 归档目录。

第五步：设计确定性规则

优先解决：

- 路径；
- 命名；
- 编号；
- 日期；
- 状态机；
- 周期任务。

第六步：编写 Prompt

只处理仍需要语义理解的内容。

第七步：设计审核边界

列出：

- 自动执行；
- 执行后审核；
- 事前确认。

第八步：设计 Dashboard

只提交真正需要用户行动的事项。

第九步：添加测试 Fixture

覆盖正常、模糊和错误输入。

第十步：创建试用实例

在测试 Vault 中运行，不直接使用正式数据。

35. 模块骨架生成命令

建议核心平台提供：


```
pkb module create reading-log
```

生成：

```
Modules/reading-log/  
├── module.yaml  
├── README.md  
├── schemas/  
├── prompts/  
├── workflows/  
├── rules/  
├── templates/  
├── dashboard/  
└── tests/
```

并提供：

```
pkb module validate reading-log  
pkb module test reading-log  
pkb module enable reading-log  
pkb module disable reading-log
```

36. 最小模块示例

一个最简单的阅读记录模块可以只包含：

```
reading-log/  
├── module.yaml  
├── schemas/reading-note.schema.yaml  
├── prompts/normalize-reading-note.md  
├── workflows/process-capture.yaml  
├── rules/paths.yaml  
├── rules/review-policy.yaml  
└── templates/quick-note.md
```

它不一定需要：

- 状态机；
- 跨模块事件；
- 自定义脚本；
- 实例 Inbox；
- 周期任务。

模块应按实际需要增加复杂度，避免“为了模块化而模块化”。

37. 当前第一批模块建议

application-tracker

优先实现：

- 状态机；
- Research Request；
- 关键字段审核；
- 到期复查；
- 比较更新。

experience-log

优先实现：

- 混乱记录规范化；
- 周月总结；
- 技能证据；
- STAR素材；
- 事实保护。

course

优先实现：

- 实例 Inbox；
- 附件伴随笔记；
- 作业编号；
- 课堂笔记；
- 文件读取等级。

journal

优先实现：

- 用户正文保护；
- 周月年总结；
- 保守链接；
- 不进行人格推断。

38. 模块开发核心原则

每个模块都应遵循：

路径优先于正文，规则优先于 Prompt，增量优先于全量，结构化输出优先于自由文本，审核优先于猜测，事件优先于模块互相修改，归档优先于删除。

一个模块的质量不取决于 Prompt 写得有多长，而取决于：

- 数据边界是否清楚；
- 自动化是否可预测；
- 不确定性是否被正确暴露；
- 是否尽量减少用户维护成本；
- 是否能够安全停用和归档；
- 是否可以在不修改核心平台的情况下独立演进。